



Contoh-Contoh Program C++

IF2210 – Semester II 2025/2026

Tim Pengajar IF2210

Tujuan

- Contoh-contoh berikut:
 - untuk membantu Anda membaca kode C++ di dunia nyata,
 - bukan untuk dipakai sebagai pola desain utama dalam mata kuliah ini.
- Fokus utama kuliah tetap: **desain objek dan tanggung jawabnya**.
- Catatan: agar muat di 1 slide, contoh-contoh dibuat *inline* (deklarasi dan definisi jadi 1).
 - Saat praktik, jangan lupa pisahkan header (.hpp) dan implementasi (.cpp) *dan* gunakan formatting yang baik.

std::vector

- Gunakan `std::vector` ketika suatu objek memiliki **sekumpulan objek lain** (koleksi).
- *Lifecycle* setiap elemen diurus otomatis.
- Sebagian kontrak:
 - `push_back(x)` → menambah elemen.
 - `size()` → jumlah elemen.
 - `v[i]` → akses elemen.
 - `std::vector<T>` = koleksi objek bertipe T.

std::string

- Gunakan `std::string` untuk merepresentasikan teks sebagai **value object**.

```
#include <string>
class Person {
public:
    Person(std::string name) : name_(std::move(name)) {}

    const std::string& name() const { return name_; }

private:
    std::string name_;
};
```

std::string

- **Makna Konseptual**
 - Nama adalah nilai (value), bukan array karakter.
 - Tidak ada `strlen`, `malloc`, atau null-termination.
 - String mengurus memorinya sendiri (RAII).
 - Bisa disalin, dipindah, dan dikembalikan seperti nilai biasa.
- **Yang Perlu Diketahui Saat Ini:**
 - Contoh inisialisasi: `std::string s = "Ali";`
 - Bisa dibandingkan, disalin, dan dikirim sebagai parameter.
 - Perlakukan seperti `int` atau `double`.

Rule of 0

- Jika anggota kelas sudah mengelola resource dengan benar, kita tidak perlu menulis operator= sama sekali.

```
#include <vector>
class Buffer {
public:
    explicit Buffer(std::size_t n = 0) : data_(n) {}
    std::size_t size() const { return data_.size(); }
private:
    std::vector<int> data_; // RAII → copy/move otomatis benar
};
```

- Biarkan `std::vector` mengurus memori.

Jika Mengelola Resource Mentah (Legacy Pattern)

- Kadang Anda akan menemui kelas yang memakai `new[]` / `delete[]`.

```
class RawBuffer {  
    public:  
        RawBuffer(std::size_t n) : size_(n), data_(new int[n]) {}  
        ~RawBuffer() { delete[] data_; }  
        RawBuffer(const RawBuffer& other)  
            : size_(other.size_), data_(new int[other.size_]) {  
            std::copy(other.data_, other.data_ + size_, data_);  
        }  
        RawBuffer& operator=(const RawBuffer& other);  
};
```

- Ini teknik C++ lama. Pahami, tapi jangan dijadikan default desain tanpa alasan kuat.

Copy Assignment yang Aman

```
RawBuffer& RawBuffer::operator=(const RawBuffer& other) {  
    if (this == &other) return *this;  
  
    int* newData = new int[other.size_];  
    std::copy(other.data_, other.data_ + other.size_, newData);  
  
    delete[] data_;  
    data_ = newData;  
    size_ = other.size_;  
    return *this;  
}
```

- Fokusnya menjaga invariant dan ownership, bukan logika domain.

Move Assignment (Memindahkan Ownership)

```
RawBuffer& operator=(RawBuffer&& other) noexcept {  
    delete[] data_;  
  
    data_ = std::exchange(other.data_, nullptr);  
    size_ = std::exchange(other.size_, 0);  
  
    return *this;  
}
```

- Resource dipindahkan, bukan disalin.

Copy-and-Swap

```
RawBuffer& operator=(RawBuffer other) noexcept {  
    std::swap(other);  
    return *this;  
}
```

- Teknik ringkas yang sering ditemui di pustaka C++.

operator* pada Fraction

- Operator aritmatika menghasilkan nilai baru.

```
class Fraction {  
public:  
    Fraction(int n, int d = 1);  
  
    Fraction operator*(const Fraction& rhs) const {  
        return Fraction(n_ * rhs.n_, d_ * rhs.d_);  
    }  
  
private:  
    int n_, d_;  
};
```

- * tidak mengubah operand.

operator*= untuk Mutasi yang Eksplisit

```
Fraction& operator*=(const Fraction& rhs) {  
    *this = *this * rhs;  
    return *this;  
}
```

- Jika perlu mengubah state (“memutasi”) objek, gunakan operator assignment-style.

Overload untuk Output (operator<<)

- Integrasi dengan ekosistem C++.

```
std::ostream& operator<<(std::ostream& out, const Fraction& f) {  
    return out << f.num() << '/' << f.den();  
}
```

- Contoh penggunaan (e.g., di main function):

```
Fraction f1(3, 4);  
Fraction f2(5, 6);
```

```
std::cout << "f1 = " << f1 << '\n';  
std::cout << "f2 = " << f2 << '\n';
```

Unary Operator (Contoh: operator-)

```
#include <iostream>
class Distance {
public:
    Distance(int feet=0, int inches=0) : feet_(feet), inches_(inches) {}
    Distance operator-() const { return Distance(-feet_, -inches_); }
private:
    int feet_;
    int inches_;
};

std::ostream& operator<<(std::ostream& os, const Distance& d) {
    return os << d.feet_ << " feet " << d.inches_ << " inches";
}
```

Unary Operator – Pemakaian

```
Distance d1(11, 10);  
Distance d2 = -d1;
```

```
std::cout << "d1 = " << d1 << '\n'; // 11 feet 10 inches  
std::cout << "d2 = " << d2 << '\n'; // -11 feet -10 inches
```

```
// Bisa di-chain seperti operasi nilai biasa  
Distance d3 = -(-d1);  
std::cout << "d3 = " << d3 << '\n';
```

Operator Kadang Ditulis sebagai Free Function

- Anda mungkin menemui ini:

```
Vector2D operator*(double k, const Vector2D& v) {  
    return Vector2D(v.x() * k, v.y() * k);  
}
```

- Variasi implementasi bahasa.